# CS1026B Assignment 4: Workout Vital Signs Tracker

Due: **Tuesday, April 7, 2026**

## Change Log

- 2026-03-23 Initial post, no changes yet.

## Table of Contents


- Late Penalty
- Academic Integrity
- Learning Objectives
- Assignment Overview
  - Part 1: File Formats
  - Part 2: Raising and Handling Exceptions
  - Part 3: Valid and Abnormal Vital Ranges
  - Part 4: Classes to Implement
    - 4.1 Song (`song.py`)
    - 4.2 Vitals (`vitals.py`)
    - 4.3 Workout (`workout.py`)
      - 4.3.1: Song-Vitals Correlation
    - 4.4 Patient (`patient.py`)
    - 4.5 Clinic (`clinic.py`)
      - 4.5.1: Using the CSV Module
  - Part 5: Interactive Menu (`main.py`)
  - Part 6: Patient Report Format

- Part 7: What to Submit
- Part 8: Starter Templates
- Non-Functional Specification and Other Details
- Assessment


## Late Penalty

Late submissions will be given a **zero grade** if you have insufficient late coupons remaining.

Any resubmissions past the due date will be considered late and will require late coupons.

Assignments will be accepted up to three days late, but **only if you have at least one late coupon remaining for each day late.** (See the course syllabus for full details on how late coupons work.)

If you submit late and do not have enough late coupons remaining, a **zero grade** will be given for the assignment (there is no partial late mark).

No action is required to submit late. Simply submit late and your late coupons will be applied by the TA during marking.

▲ Back to Top

## Academic Integrity

By submitting this assignment, you agree that you have read and understood the academic dishonesty policy for this course covered in both the course syllabus and the academic dishonesty document. This includes:

- Assignments must be completed individually, without any help from others.
- No copying solutions, formulas, files, or written answers from other students.
- No sharing of any part of your work with other students.

- No unauthorized use of generative AI tools (e.g., ChatGPT, Copilot, etc.). You can not use generative AI for this assignment.

> ❗ Suspected violations will be reported to the integrity committee and receive a zero grade.

▲ Back to Top

## ◎ Learning Objectives

By completing this assignment you will demonstrate that you can:

1. Define Python classes with private attributes, getters, and setters.
2. Use lists, dictionaries, tuples, and sets where each is the natural fit.
3. Implement object comparison using `__eq__`, `__lt__`, and `__gt__`.
4. Use `__repr__` and `__str__` to provide meaningful string representations of objects.
5. Define and raise custom exception classes.
6. Handle exceptions using `try` and `except`.
7. Parse a CSV file and use it to populate objects.
8. Write a human-readable plain text report from object data.
9. Build a robust interactive menu that handles invalid input gracefully.

▲ Back to Top

## Assignment Instructions

> ℹ Make sure that you read over the Non-Functional Specification before you start coding. It contains requirements for comments, variable names, and style you need to follow.

For this assignment, you are building a vital sign tracking application (TrackFit) for patients at a fitness clinic. The application will track information from patient workout sessions. Each patient will have one or more sessions that include a playlist of songs alongside vital sign readings that were recorded during a patient's workout.

- `song.py`
- `vitals.py`
- `workout.py`
- `patient.py`
- `clinic.py`
- `main.py`

## Part 1: File Formats

Your program will work with two file formats: a CSV file for importing seed data, and a plain text file for exporting patient reports. The formats are described below.

### patients.csv

Three CSV files are provided with this assignment: `simple_patients_1.csv`, `simple_patients_2.csv`, and `patients.csv`. You do not need to create these files.

- **`simple_patients_1.csv`**: One patient, one session, three songs, four vitals. All values are valid. Use this first to confirm your CSV loading code works.
- **`simple_patients_2.csv`**: One patient, two sessions. Contains one deliberately invalid vital reading (heart rate of 400) to test that `load_csv()` catches the `ValueError` raised by `Vitals`, prints a message, and continues loading rather than crashing.
- **`patients.csv`**: Four patients with two or more sessions each, plus two edge cases: P005 (Samantha Baker) has no sessions, and P006 (Cameron Frye) has a session with no vitals logged yet. Use this file to test your full program once the simpler files work.

The CSV uses multiple record types in a single file. The first column of every row is a `record_type` field that tells you what kind of data the row contains. Every row has exactly 16 columns. Blank columns are left empty but still appear in the row. The column layout is:

| record_type | Columns used (others are left blank) |
|---|---|
| patient | patient_id, name, age, conditions (separated by \| if multiple) |
| session | patient_id, session_id, date |
| song | patient_id, session_id, title, artist, bpm, genre |
| vital | patient_id, session_id, heart_rate, bp_systolic, bp_diastolic, o2_saturation, timestamp |

Example rows (as they appear in the provided files):

```
record_type,patient_id,name,age,conditions,session_id,date,title,artis
patient,P001,Kevin McCallister,29,asthma,,,,,,,,,,
session,P001,,,,session_001,2024-03-10,,,,,,,,
song,P001,,,,session_001,,Eye of the Tiger,Survivor,109,rock,,,,,
vital,P001,,,,session_001,,,,,,,88,118,76,98.2,2024-03-10 09:00
```

**Reading the example:** Each row always has all 16 columns. Which columns contain data depends on the `record_type`. For example, a `patient` row uses `patient_id`, `name`, `age`, and `conditions`, leaving the rest blank. A `vital` row uses `heart_rate`, `bp_systolic`, `bp_diastolic`, `o2_saturation`, and `timestamp`, leaving the patient name and session date blank. Scroll right in the example above to see all 16 columns. Your `load_csv()` method must handle this exact format.

### report_<patient_id>.txt

A plain text file exported for one patient. It is formatted so that a trainer with no programming knowledge can easily read it. The required contents and an example are shown in Part 6.

▲ Back to Top

## Part 2: Raising and Handling Exceptions

Your program must raise and handle Python's built-in `ValueError` in two specific situations:

| Where | When to raise `ValueError` |
|---|---|

| | |
|---|---|
| `Vitals.__init__()` | When any vital value is outside the valid ranges defined in Part 3. Raise before the object is created. |
| `Clinic.get_patient()` | When the given `patient_id` does not exist in the patients dictionary. |

Wherever these errors can occur, the calling code must catch them using `try` and `except` and print a helpful message rather than letting the program crash. For example:

```
try:
    v = Vitals(400, (118, 76), 98.2, "2024-03-10 09:30")
except ValueError as e:
    print("Error:", e)
```

▲ Back to Top

## Part 3: Valid and Abnormal Vital Ranges

The `Vitals` class must enforce valid ranges when it is created. If any value is outside the valid range, a `ValueError` must be raised. Separately, a reading is considered clinically *abnormal* if it falls outside the normal range. These two thresholds are different.

| Vital sign | Valid range (raises error if outside) | Normal range | Abnormal if… |
|---|---|---|---|
| Heart rate | 1 - 300 bpm | 60 - 100 bpm | < 60 or > 100 |
| Systolic BP | 40 - 300 mmHg | 90 - 140 mmHg | < 90 or > 140 |
| Diastolic BP | 20 - 200 mmHg | 60 - 90 mmHg | < 60 or > 90 |
| O2 saturation | 50.0 - 100.0 % | 95.0 - 100.0 % | < 95.0 % |

**Valid vs Abnormal**

The **valid range** is the range of physiologically possible values. Anything outside this range raises a `ValueError` immediately. The **normal range** is the clinically healthy range. A reading can be valid (possible) but still abnormal (outside the healthy range). For example, a heart rate of 45 is valid (a real reading) but abnormal (below the healthy range of 60-100).

▲ Back to Top

## Part 4: Classes to Implement

You must implement five classes across five separate Python files. Each class is described below. You are provided the class, attribute, and method names in the provided templates. You must write all of the logic yourself. See Part 8 for templates to build from.

**Tip**

Build and test one class at a time in the order they are presented in this section: `Song` and `Vitals` first, then `Workout`, then `Patient`, then `Clinic`. Do not start on the menu in `main` until your classes work correctly. Code has been provided to test as you go, and you are encouraged to create your own test file using the test snippets provided. It is possible that some methods will not have tests provided, so feel free to test those yourself using the approach presented in this document.

All attributes and methods in this assignment are **instance attributes** and **instance methods**. Do not use class attributes nor class methods.

### 4.1 Song

The `Song` class represents a single song. Implement it in `song.py`.

**Attributes:**

| Attribute | Type | Description |
|---|---|---|
| `_title` | `str` | Name of the song |
| `_artist` | `str` | Artist or band name |
| `_bpm` | `int` | Beats per minute (tempo) |
| `_genre` | `str` | Music genre (e.g., “rock”, “pop”, “hip-hop”) |

**Methods:**

| Method | Returns | Description |
|---|---|---|
| `__init__(self, title, artist, bpm, genre)` | `None` | Initializes all four attributes. |
| `get_title(self)` | `str` | Returns the song title. |

| | | |
|---|---|---|
| `get_artist(self)` | `str` | Returns the artist name. |
| `get_bpm(self)` | `int` | Returns the tempo in beats per minute. |
| `get_genre(self)` | `str` | Returns the genre. |
| `__str__(self)` | `str` | Returns a readable string for the object. Format: `"<title>" by <artist> (<bpm> bpm, <genre>)` |
| `__repr__(self)` | `str` | Returns a string that looks like the constructor call. Example: `Song('Scenario', 'A Tribe Called Quest', 93, 'hip-hop')` |
| `__eq__(self, other)` | `bool` | Two `Song` objects are equal if their `title` and `artist` both match. Comparison is case-sensitive. |

## Testing the Song Class

**Example:**

```
from song import Song

s1 = Song("Scenario", "A Tribe Called Quest", 93, "hip-hop")
s2 = Song("Scenario", "A Tribe Called Quest", 99, "hip-hop")
s3 = Song("Sabotage", "The Beastie Boys", 93, "hip-hop")

print(s1)
print(repr(s1))
print(s1.get_title())
print(s1.get_artist())
print(s1.get_bpm())
print(s1.get_genre())
print(s1 == s2)
print(s1 == s3)
```

**Output:**

```
"Scenario" by A Tribe Called Quest (93 bpm, hip-hop)
Song('Scenario', 'A Tribe Called Quest', 93, 'hip-hop')
```

```
Scenario
A Tribe Called Quest
93
hip-hop
True
False
```

▲ Back to Top

---

## 4.2 Vitals

The `Vitals` class represents a set of vital signs recorded at a specific moment. Implement it in `vitals.py`. All attributes are private. Once created, a vitals reading shouldn't change, so this class has accessors (i.e., getters) but no mutators (i.e., setters). When a `Vitals` object is created, its potential attributes must be validated against the valid ranges described in the table in Part 3. If any value is outside the **valid range** (remember, an abnormal reading is something different), then a `ValueError` must be raised before the `Vitals` object is created. This error should prevent the creation of the object and let the user know why the data was invalid. Check values in the following order: heart rate, systolic BP, diastolic BP, O2 saturation.

**Attributes:**

| Attribute | Type | Description |
|---|---|---|
| `_heart_rate` | `int` | Heart rate in beats per minute |
| `_blood_pressure` | `tuple` | A (systolic, diastolic) pair (e.g., `(118, 76)`). |
| `_o2_saturation` | `float` | Blood oxygen saturation as a percentage (e.g., `97.5`). |
| `_timestamp` | `str` | When the reading was taken (e.g., `"2024-03-10 09:30"`). |

**Why a tuple?**

Blood pressure is always a pair of values. While it is read and stored as one attribute, `_blood_pressure`, this attribute is actually a tuple composed of two elements. The order of the elements is fixed (systolic first, diastolic second), and the values should not change once recorded. A tuple communicates all of this directly.

**Methods:**

| Method | Returns | Description |
|---|---|---|
| `__init__(self, heart_rate, blood_pressure, o2_saturation, timestamp)` | `None` | Validates all values (raises `ValueError` if any are outside the valid range), then initializes all private attributes. `_blood_pressure` must be stored as a tuple. |
| `get_heart_rate(self)` | `int` | Returns the heart rate. |
| `get_blood_pressure(self)` | `tuple` | Returns the blood pressure tuple. |
| `get_o2_saturation(self)` | `float` | Returns the O2 saturation. |
| `get_timestamp(self)` | `str` | Returns the timestamp. |
| `is_abnormal(self)` | `bool` | Returns `True` if any value is outside the normal range Part 3, `False` otherwise. |
| `__str__(self)` | `str` | Returns a readable string. Format: `HR: <hr> bpm \| BP: <sys>/<dia> \| O2: <o2>% \| <timestamp>` |
| `__repr__(self)` | `str` | Returns a string that looks like the constructor call. For example, `Vitals(88, (118, 76), 98.2, '2024-03-10 09:00')` |
| `__eq__(self, other)` | `bool` | Returns `True` if all four values match. |
| `__lt__(self, other)` | `bool` | Returns `True` if this reading's heart rate is lower than the other's. |
| `__gt__(self, other)` | `bool` | Returns `True` if this reading's heart rate is greater than the other's. |

The error messages for `ValueError` must follow this pattern exactly:

| Invalid value | Error message |
|---|---|
| Heart rate | `Invalid heart rate: 400. Must be between 1 and 300.` |
| Systolic blood pressure | `Invalid systolic blood pressure: 350. Must be between 40 and 300.` |

| | |
|---|---|
| Diastolic blood pressure | `Invalid diastolic blood pressure: 15. Must be between 20 and 200.` |
| O2 saturation | `Invalid O2 saturation: 49.0. Must be between 50.0 and 100.0.` |

## Testing the Vitals Class

### Creates a `Vitals` object and print it:

```
from vitals import Vitals

v = Vitals(88, (118, 76), 98.2, "2024-03-10 09:00")
print(v)
```

### Output:

```
HR: 88 bpm | BP: 118/76 | O2: 98.2% | 2024-03-10 09:00
```

### Create a `Vitals` object and prints its `__repr__`:

```
from vitals import Vitals

v = Vitals(88, (118, 76), 98.2, "2024-03-10 09:00")
print(repr(v))
```

### Output:

```
Vitals(88, (118, 76), 98.2, '2024-03-10 09:00')
```

### Tests four readings. One normal reading and three abnormal readings (low heart rate, high blood pressure, low O2):

```
from vitals import Vitals

v_normal  = Vitals(88, (118, 76), 98.2, "2024-03-10 09:00")
v_low_hr  = Vitals(45, (118, 76), 98.2, "2024-03-10 09:15")
v_high_bp = Vitals(88, (145, 76), 98.2, "2024-03-10 09:30")
```

```
v_low_o2  = Vitals(88, (118, 76), 93.5, "2024-03-10 09:45")

print(v_normal.is_abnormal())
print(v_low_hr.is_abnormal())
print(v_high_bp.is_abnormal())
print(v_low_o2.is_abnormal())
```

**Output:**

```
False
True
True
True
```

**Two identical readings should be equal, while two different readings should not be equal:**

```
from vitals import Vitals

v1 = Vitals(88, (118, 76), 98.2, "2024-03-10 09:00")
v2 = Vitals(88, (118, 76), 98.2, "2024-03-10 09:00")
v3 = Vitals(95, (125, 80), 97.5, "2024-03-10 09:15")

print(v1 == v2)
print(v1 == v3)
```

**Output:**

```
True
False
```

**Comparison based on heart rate. v1 has HR 88, and v2 has HR 95:**

```
from vitals import Vitals

v1 = Vitals(88, (118, 76), 98.2, "2024-03-10 09:00")
v2 = Vitals(95, (125, 80), 97.5, "2024-03-10 09:15")

print(v1 < v2)

```

```
print(v2 > v1)
print(v1 > v2)
```

**Output:**

```
True
True
False
```

**Each of the four invalid cases should raise a `ValueError` with the exact message shown.**

```
from vitals import Vitals

try:
    Vitals(400, (118, 76), 98.2, "2024-03-10 09:00")
except ValueError as e:
    print(e)

try:
    Vitals(88, (350, 76), 98.2, "2024-03-10 09:00")
except ValueError as e:
    print(e)

try:
    Vitals(88, (118, 15), 98.2, "2024-03-10 09:00")
except ValueError as e:
    print(e)

try:
    Vitals(88, (118, 76), 49.0, "2024-03-10 09:00")
except ValueError as e:
    print(e)
```

**Output:**

```
Invalid heart rate: 400. Must be between 1 and 300.
Invalid systolic blood pressure: 350. Must be between 40 and 300.
Invalid diastolic blood pressure: 15. Must be between 20 and 200.
Invalid O2 saturation: 49.0. Must be between 50.0 and 100.0.
```

▲ Back to Top

## 4.3 Workout

The `Workout` class represents a single workout session. Implement it in `workout.py`.

**Attributes:**

| Attribute | Type | Description |
|---|---|---|
| `_session_id` | `str` | Unique identifier (e.g., `"session_001"`) |
| `_date` | `str` | Date of the session (e.g., `"2024-03-10"`). |
| `_playlist` | `list` | Ordered list of `Song` objects. |
| `_log` | `list` | List of `Vitals` objects, stored in chronological order. |

⚠ **Important:** `_playlist` is a list of `Song` objects and `_log` is a list of `Vitals` objects. They are not lists of strings nor of integers. When you loop through them you will need to call getter methods to access their values. Each of the classes builds upon the previous, so you must remember to check how to access an object's attributes via the methods you have already created. For example, to get a heart rate reading from the `Workout` object's `log` of `Vitals` objects, you need to call `reading.get_heart_rate()`, where `reading` is one of the `Vitals` objects stored in the `log`.

**Methods:**

| Method | Returns | Description |
|---|---|---|
| `__init__(self, session_id, date)` | `None` | Initializes private attributes. `_playlist` and `_log` start as empty lists. |
| `get_session_id(self)` | `str` | Returns the session ID. |
| `get_date(self)` | `str` | Returns the session date. |
| `get_playlist(self)` | `list` | Returns a copy of the playlist so the internal list cannot be modified from outside. |

| | | |
|---|---|---|
| `get_log(self)` | `list` | Returns a copy of the vitals log so the internal list cannot be modified from outside. |
| `add_song(self, song)` | `None` | Adds a `Song` object to the playlist. |
| `add_vital(self, vitals)` | `None` | Adds a `Vitals` object to the log. |
| `get_average_heart_rate(self)` | `float` or `None` | Returns the average heart rate across all readings. Returns `None` if the log is empty. |
| `get_unique_genres(self)` | `set` | Returns a set of unique genre strings from all songs in the playlist. |
| `get_abnormal_readings(self)` | `list` | Returns a list of all `Vitals` objects for which `is_abnormal()` returns `True`. |
| `get_segments(self)` | `list` or `None` | Divides the vitals log into equal segments, one per song in the playlist, using the positional approximation described in Part 4.3.1. Returns a list of `(Song, [Vitals])` tuples. Returns `None` if the playlist or log is empty. Used by `get_peak_heart_rate_song()` and by `Clinic` methods that need segment data. |
| `get_peak_heart_rate_song(self)` | `Song` or `None` | Returns the `Song` with the highest average heart rate segment (using `get_segments()`). Returns `None` if playlist or log is empty. |
| `__str__(self)` | `str` | Returns a readable string. Format: `" <session_id> on <date>: <n> songs, <n> readings"`. Example: `session_001 on 2024-03-10: 4 songs, 4 readings` |
| `__repr__(self)` | `str` | Returns a string that looks like the constructor call. Example: `Workout('session_001', '2024-03-10')` |
| `__eq__(self, other)` | `bool` | Two `Workout` objects are equal if their `session_id` matches. |

## Testing the Workout Class

**Two readings averaging 85.0. An empty session returns None:**

```
from song import Song
from vitals import Vitals
from workout import Workout

w = Workout("session_001", "2024-03-10")
w.add_song(Song("Eye of the Tiger", "Survivor", 109, "rock"))
w.add_vital(Vitals(80, (116, 75), 98.2, "2024-03-10 09:00"))
w.add_vital(Vitals(90, (122, 78), 97.8, "2024-03-10 09:15"))

print(w.get_average_heart_rate())

empty = Workout("session_empty", "2024-03-10")
print(empty.get_average_heart_rate())
```

**Output:**

```
85.0
None
```

**Three songs: two rock, one pop. The set contains only the unique genres:**

```
from song import Song
from vitals import Vitals
from workout import Workout

w = Workout("session_001", "2024-03-10")
w.add_song(Song("Eye of the Tiger", "Survivor", 109, "rock"))
w.add_song(Song("Mr. Brightside", "The Killers", 148, "rock"))
w.add_song(Song("Don't Stop the Music", "Rihanna", 171, "pop"))

print(w.get_unique_genres())
```

**Output:**

```
{'rock', 'pop'}
```

> ⚠ Reminder: sets are unordered, so the order of elements may vary.

**Two readings: one normal (HR 88), one abnormal (HR 45). The list contains only the abnormal one:**

```
from song import Song
from vitals import Vitals
from workout import Workout

w = Workout("session_001", "2024-03-10")
w.add_song(Song("Eye of the Tiger", "Survivor", 109, "rock"))
w.add_vital(Vitals(88, (118, 76), 98.2, "2024-03-10 09:00"))
w.add_vital(Vitals(45, (118, 76), 98.2, "2024-03-10 09:15"))

print(len(w.get_abnormal_readings()))
print(w.get_abnormal_readings()[0])
```

**Output:**

```
1
HR: 45 bpm | BP: 118/76 | O2: 98.2% | 2024-03-10 09:15
```

**Three readings and three songs. There is an even split, so one reading per song:**

Select Code Download

```
from song import Song
from vitals import Vitals
from workout import Workout

w = Workout("session_001", "2024-03-10")
w.add_song(Song("Eye of the Tiger", "Survivor", 109, "rock"))
w.add_song(Song("Don't Stop the Music", "Rihanna", 171, "pop"))
w.add_song(Song("Scenario", "A Tribe Called Quest",  93, "hip-hop"))
w.add_vital(Vitals(80, (116, 75), 98.2, "2024-03-10 09:00"))
w.add_vital(Vitals(97, (128, 82), 97.3, "2024-03-10 09:15"))
w.add_vital(Vitals(84, (117, 76), 98.1, "2024-03-10 09:30"))

print("Even split (3 songs, 3 readings):")
for song, readings in w.get_segments():
    print("  " + song.get_title() + ": " + str(len(readings)) + " read

```

```
    for v in readings:
        print("    " + str(v))
```

**Output:**

```
Even split (3 songs, 3 readings):
  Eye of the Tiger: 1 reading(s)
    HR: 80 bpm | BP: 116/75 | O2: 98.2% | 2024-03-10 09:00
  Don't Stop the Music: 1 reading(s)
    HR: 97 bpm | BP: 128/82 | O2: 97.3% | 2024-03-10 09:15
  Scenario: 1 reading(s)
    HR: 84 bpm | BP: 117/76 | O2: 98.1% | 2024-03-10 09:30
```

**Three readings, but only two songs. There is an uneven split, so the first song gets the extra reading:**

```
from song import Song
from vitals import Vitals
from workout import Workout

w = Workout("session_002", "2024-03-10")
w.add_song(Song("Eye of the Tiger", "Survivor", 109, "rock"))
w.add_song(Song("Scenario", "A Tribe Called Quest",  93, "hip-hop"))
w.add_vital(Vitals(80, (116, 75), 98.2, "2024-03-10 09:00"))
w.add_vital(Vitals(97, (128, 82), 97.3, "2024-03-10 09:15"))
w.add_vital(Vitals(84, (117, 76), 98.1, "2024-03-10 09:30"))

print("Uneven split (2 songs, 3 readings):")
for song, readings in w.get_segments():
    print("  " + song.get_title() + ": " + str(len(readings)) + " read
    for v in readings:
        print("    " + str(v))
```

**Output:**

```
Uneven split (2 songs, 3 readings):
  Eye of the Tiger: 2 reading(s)
    HR: 80 bpm | BP: 116/75 | O2: 98.2% | 2024-03-10 09:00
    HR: 97 bpm | BP: 128/82 | O2: 97.3% | 2024-03-10 09:15
```

```
  Scenario: 1 reading(s)
    HR: 84 bpm | BP: 117/76 | O2: 98.1% | 2024-03-10 09:30
```

**Peak heart rate. Three songs with one reading each. An empty session returns None:**

```python
from song import Song
from vitals import Vitals
from workout import Workout

w = Workout("session_001", "2024-03-10")
w.add_song(Song("Eye of the Tiger", "Survivor", 109, "rock"))
w.add_song(Song("Don't Stop the Music", "Rihanna", 171, "pop"))
w.add_song(Song("Scenario", "A Tribe Called Quest", 93, "hip-hop"))
w.add_vital(Vitals(80, (116, 75), 98.2, "2024-03-10 09:00"))
w.add_vital(Vitals(97, (128, 82), 97.3, "2024-03-10 09:15"))
w.add_vital(Vitals(84, (117, 76), 98.1, "2024-03-10 09:30"))

print(w.get_peak_heart_rate_song())

empty = Workout("session_empty", "2024-03-10")
print(empty.get_peak_heart_rate_song())
```

**Output:**

```
Don't Stop the Music by Rihanna (171 bpm, pop)
None
```

▲ Back to Top

---

## Part 4.3.1: Song-Vitals Correlation

The `get_peak_heart_rate_song()` method in `Workout` must return the `Song` associated with the highest average heart rate. Since the application does not track exactly which song was playing at each moment, use the following positional approximation to pretend that you know what song is playing during a specific reading.

Divide the `vitals_log` into equal segments, one segment per song in the playlist. Assign each segment to the corresponding song. Compute the average heart rate for each segment and return the `Song` with the highest average.

If the number of readings does not divide evenly by the number of songs, distribute the remainder across the first songs (each of those songs gets one extra reading). For example, with three songs and nine readings, Song 0 gets readings 0-2, Song 1 gets readings 3-5, and Song 2 gets readings 6-8. With three songs and 10 readings, Song 0 gets readings 0-3, and Song 1 gets 4-6 and Song 2 gets 7-9. If there were 11 readings, then Song 0 would get four readings, Song 1 would get four readings, and Song 3 would get three readings.

Figure 1 provides an example of both the even and uneven cases. In the even case, there are three songs and three readings, so each song gets one reading. In the uneven case, there are only two songs for three readings, so Song 0 gets two readings and Song 1 gets one reading.

> This is an intentional approximation. The goal is to practice writing a method that combines two lists in a non-trivial way. You are not expected to implement real-time playback tracking. The same approximation is used by `get_genre_hr_averages()` in `Clinic`.

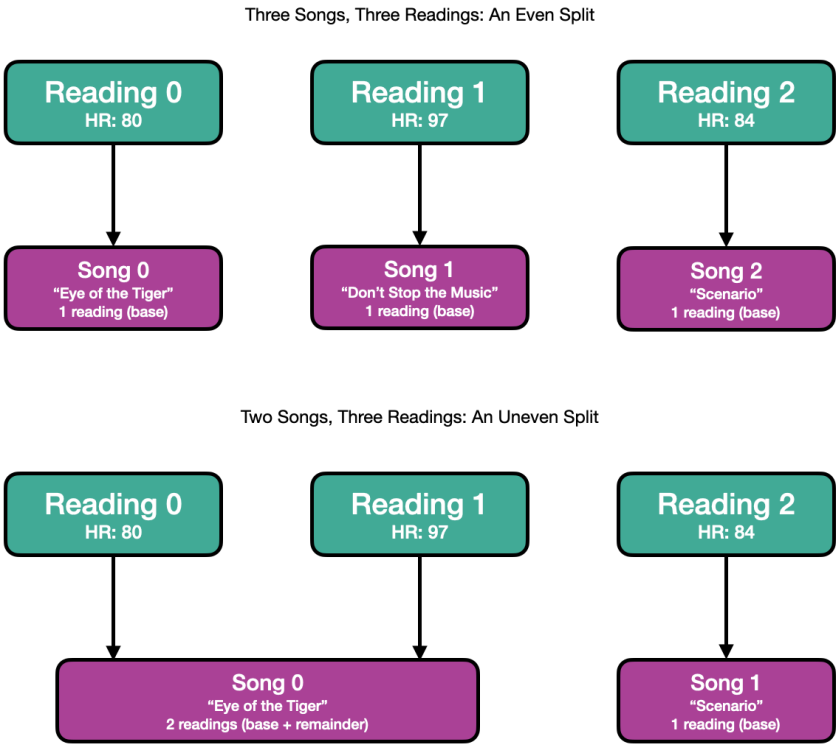


Figure 1: Positional approximation used by `get_segments()`

▲ Back to Top

## 4.4 Patient

The `Patient` class represents a clinic patient and owns a collection of `Workout` objects. Implement it in `patient.py`.

### Attributes:

| Attribute | Type | Description |
|---|---|---|
| `_patient_id` | `str` | Unique identifier (e.g., `"P001"`). |
| `_name` | `str` | Full name of the patient. |
| `_age` | `int` | Age in years. |
| `_conditions` | `set` | Set of known medical conditions (e.g., `{"hypertension", "asthma"}`). |
| `_sessions` | `dict` | Maps `session_id` (str) to `Workout` object. |

**Why a set for conditions?**

A patient either has a condition or they don't. Duplicates are meaningless and order does not matter. A set models this and makes membership testing efficient.

### Methods:

| Method | Returns | Description |
|---|---|---|
| `__init__(self, patient_id, name, age, conditions)` | `None` | Initializes all private attributes. `conditions` must be stored as a `set`, even if passed as a list. |
| `get_patient_id(self)` | `str` | Returns the patient ID. |
| `get_name(self)` | `str` | Returns the patient name. |
| `get_age(self)` | `int` | Returns the patient age. |
| `get_conditions(self)` | `set` | Returns a copy of the conditions set. |
| `get_sessions(self)` | `dict` | Returns a copy of the sessions dictionary. |
| `set_name(self, name)` | `None` | Updates the patient's name. |
| `set_age(self, age)` | `None` | Updates the patient's age. Raises `ValueError` if age is not a positive integer. |

| | | |
|---|---|---|
| `add_condition(self, condition)` | `None` | Adds a condition to the patient's conditions set. |
| `remove_condition(self, condition)` | `None` | Removes a condition from the patient's conditions set. Does nothing if the condition is not present. Hint: check whether the condition exists before calling `remove()` on the set, or use the set's `discard()` method which does the same thing safely. |
| `add_session(self, session)` | `None` | Adds a `Workout` to the sessions dictionary using its `session_id` as the key. |
| `get_session(self, session_id)` | `Workout` or `None` | Returns the `Workout` with the given `session_id`, or `None` if not found. |
| `get_overall_avg_heart_rate(self)` | `float` or `None` | Returns the average heart rate across all readings in all sessions. Returns `None` if no readings exist. Hint: loop through all sessions, then for each session, loop through its log. |
| `get_heart_rate_trend(self)` | `list` | Returns a list of `(date, avg_heart_rate)` tuples sorted by date (oldest first). Sessions with no vital readings are skipped. Hint: use a method that you already implemented in `Workout`; the method returns `None` for empty sessions, which is how you know to skip them. |
| `get_worst_session(self)` | `Workout` or `None` | Returns the `Workout` with the largest number of abnormal readings (i.e. readings for which `is_abnormal()` returns `True`). If two sessions are tied, return the one that is found first. If all sessions have zero abnormal readings, return the first session. Returns `None` if no sessions exist. Note: if all sessions have zero abnormal readings, the method should still return a session. Hint: use code that you already implemented in `Workout` to help here. |

| | | |
|---|---|---|
| `get_top_genre(self)` | `str` or `None` | Returns the most frequently occurring genre across all sessions. Returns `None` if no songs exist. |
| `__str__(self)` | `str` | Returns a readable string. Format: "`<name> (<id>) \| Age: <age> \| Conditions: <conditions>`". Conditions should be sorted alphabetically and separated by commas. Examples: `Kevin McCallister (P001) \| Age: 29 \| Conditions: asthma` OR `Buck Russell (P002) \| Age: 41 \| Conditions: diabetes, hypertension`. Hint: you can use `join` and `sorted` to produce the correctly sorted, comma-separated string. |
| `__repr__(self)` | `str` | Returns a string that looks like the constructor call. Example: `Patient('P001', 'Kevin McCallister', 29, {'asthma'})` |
| `__eq__(self, other)` | `bool` | Two `Patient` objects are equal if their `patient_id` matches. |

## Testing the Patient Class

### Two sessions with two readings each. A patient with no sessions returns None.

```
from song import Song
from vitals import Vitals
from workout import Workout
from patient import Patient

p = Patient("P001", "Kevin McCallister", 29, {"asthma"})

s1 = Workout("session_001", "2024-03-10")
s1.add_song(Song("Eye of the Tiger", "Survivor", 109, "rock"))
s1.add_vital(Vitals(80, (116, 75), 98.2, "2024-03-10 09:00"))
s1.add_vital(Vitals(90, (122, 78), 97.8, "2024-03-10 09:15"))

```

```
s2 = Workout("session_002", "2024-03-17")
s2.add_song(Song("Shake It Off", "Taylor Swift", 160, "pop"))
s2.add_vital(Vitals(84, (117, 76), 98.3, "2024-03-17 09:00"))
s2.add_vital(Vitals(88, (119, 77), 98.1, "2024-03-17 09:15"))

p.add_session(s1)
p.add_session(s2)

print(p.get_overall_avg_heart_rate())

empty_p = Patient("P999", "Nobody", 30, set())
print(empty_p.get_overall_avg_heart_rate())
```

**Output:**

```
85.5
None
```

**Two sessions. Returns a sorted list of (date, average HR) tuples in ascending chronological order.**

```
from song import Song
from vitals import Vitals
from workout import Workout
from patient import Patient

p = Patient("P001", "Kevin McCallister", 29, {"asthma"})

s1 = Workout("session_001", "2024-03-10")
s1.add_song(Song("Eye of the Tiger", "Survivor", 109, "rock"))
s1.add_vital(Vitals(80, (116, 75), 98.2, "2024-03-10 09:00"))
s1.add_vital(Vitals(90, (122, 78), 97.8, "2024-03-10 09:15"))

s2 = Workout("session_002", "2024-03-17")
s2.add_song(Song("Shake It Off", "Taylor Swift", 160, "pop"))
s2.add_vital(Vitals(84, (117, 76), 98.3, "2024-03-17 09:00"))
s2.add_vital(Vitals(88, (119, 77), 98.1, "2024-03-17 09:15"))

p.add_session(s1)
p.add_session(s2)

```

```
for date, avg in p.get_heart_rate_trend():
    print(date + ": " + str(round(avg, 1)) + " bpm")
```

**Output:**

```
2024-03-10: 85.0 bpm
2024-03-17: 86.0 bpm
```

**The most (i.e, highest count of) abnormal readings. A patient with no sessions returns None.**

```
from song import Song
from vitals import Vitals
from workout import Workout
from patient import Patient

p = Patient("P001", "Kevin MacAllister", 29, {"asthma"})

s1 = Workout("session_001", "2024-03-10")
s1.add_song(Song("Eye of the Tiger", "Survivor", 109, "rock"))
s1.add_vital(Vitals(88, (118, 76), 98.2, "2024-03-10 09:00"))

s2 = Workout("session_002", "2024-03-17")
s2.add_song(Song("Scenario", "A Tribe Called Quest", 93, "hip-hop"))
s2.add_vital(Vitals(45, (88, 58), 93.0, "2024-03-17 09:00"))
s2.add_vital(Vitals(88, (118, 76), 98.2, "2024-03-17 09:15"))

p.add_session(s1)
p.add_session(s2)

print(p.get_worst_session().get_session_id())

empty_p = Patient("P999", "Nobody", 30, set())
print(empty_p.get_worst_session())
```

**Output:**

```
session_002
None
```

**The top genre. A patient with no songs returns None.**

```
from song import Song
from vitals import Vitals
from workout import Workout
from patient import Patient

p = Patient("P001", "Kevin McCallister", 29, {"asthma"})

s1 = Workout("session_001", "2024-03-10")
s1.add_song(Song("Eye of the Tiger", "Survivor", 109, "rock"))
s1.add_song(Song("Mr. Brightside", "The Killers", 148, "rock"))
s1.add_song(Song("Don't Stop the Music", "Rihanna", 171, "pop"))
s1.add_vital(Vitals(88, (118, 76), 98.2, "2024-03-10 09:00"))
p.add_session(s1)

print(p.get_top_genre())

empty_p = Patient("P999", "Nobody", 30, set())
print(empty_p.get_top_genre())
```

**Output:**

```
rock
None
```

▲ Back to Top

---

## 4.5 Clinic

The `Clinic` class is the top-level manager. It holds the full patient roster and is responsible for all file input and output. Implement it in `clinic.py`.

**Attributes:**

| Attribute | Type | Description |
|---|---|---|
| `_name` | `str` | Name of the clinic. |
| `_patients` | `dict` | Maps `patient_id` (str) to `Patient` object. |

### Methods:

| Method | Returns | Description |
|---|---|---|
| `__init__(self, name)` | `None` | Initializes `_name` and an empty `_patients` dictionary. |
| `get_name(self)` | `str` | Returns the clinic name. |
| `set_name(self, name)` | `None` | Updates the clinic name. |
| `get_patients(self)` | `dict` | Returns a copy of the patients dictionary. |
| `add_patient(self, patient)` | `None` | Adds a `Patient` to the patients dictionary using `patient_id` as the key. |
| `get_patient(self, patient_id)` | `Patient` | Returns the `Patient` with the given `patient_id`. Raises `ValueError` if not found. The error message must follow this format: `"Patient not found: P999"` |
| `search_patients_by_name(self, name)` | `list` | Returns a list of `Patient` objects whose name contains the given string (case-insensitive, partial match). Returns an empty list if none found. |
| `get_patients_with_condition(self, condition)` | `set` | Returns a set of `patient_id` strings for all patients who have the given condition. Returns an empty set if none found. Hint: you can use `issuperset()` or `issubset()` for a set-based solution. |
| `get_all_conditions(self)` | `set` | Returns the union of all patients' condition sets. Returns an empty set if no conditions exist. |
| `get_patients_with_conditions(self, conditions)` | `set` | Takes a set (or list) of condition strings. Returns a set of `patient_id` strings for every patient whose conditions set contains **all** of the given conditions. Returns an empty set if none match. |

| | | |
|---|---|---|
| `get_highest_avg_hr_patient(self)` | `Patient` or `None` | Returns the `Patient` with the highest overall average heart rate. If two patients are equally high, return the first one found. Returns `None` if no patients or no readings exist. |
| `get_genre_hr_averages(self)` | `dict` | Returns a dictionary where each key is a genre string (e.g., `"rock"`, `"pop"`) and each value is the average heart rate recorded across all sessions clinic-wide where that genre appeared. For each session, call `get_segments()` to get the `(Song, [Vitals])` pairs, calculate the average heart rate for each segment, and accumulate the total by genre. At the end, divide each genre's total by the number of segments it appeared in. Example result: `{"rock": 87.3, "pop": 89.0, "hip-hop": 83.3}`. Skip sessions where `get_segments()` returns `None`. |
| `load_csv(self, file_name)` | `None` | Reads the given CSV file and populates the clinic. For each row, check the `record_type` field and create the appropriate object (`Patient`, `Workout`, `Song`, or `Vitals`). If a vital reading raises a `ValueError`, print a message and continue loading rather than stopping. See the hint below in Part 4.5.1 for guidance on using Python's `csv` module. |
| `export_patient_report(self, patient_id, file_name)` | `None` | Exports a plain text report for the given patient. Raises `ValueError` if the patient does not exist. |
| `__str__(self)` | `str` | Returns a readable string. Format: `"<name> \| <n> patients"`. Example: `FitTrack Clinic \| 4 patients` |
| `__repr__(self)` | `str` | Returns a string that looks like the constructor call. Example: `Clinic('FitTrack Clinic')` |

| | | |
|---|---|---|
| `__eq__(self, other)` | `bool` | Two `Clinic` objects are equal if their name matches. Comparison is case-sensitive. |

## Part 4.5.1: Using the CSV Module

You may read the provided CSV files however you'd like, but the built-in `csv` module can make things quite a bit simpler for you.

**Using the csv module**

Python's built-in `csv` module makes it easy to read CSV files. Use `csv.DictReader` to read each row as a dictionary where the keys are the column headers. Check `row["record_type"]` first to determine what kind of object to create, then read the relevant columns for that type. Always call `.strip()` on values to remove any extra whitespace. Conditions are stored as a `set` and separated by `|` in the CSV.

```python
with open(file_name, newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        record_type = row["record_type"].strip()
        if record_type == "patient":
            patient_id = row["patient_id"].strip()
            name = row["name"].strip()
            age = int(row["age"].strip())
            cond_str = row["conditions"].strip()
            conditions = set()
            for c in cond_str.split("|"):
                if c.strip() != "":
                    conditions.add(c.strip())
            new_patient = Patient(patient_id, name, age, conditions)
            self.add_patient(new_patient)
```

⚠ The above is just a snippet to give you an idea of how to use `csv.DictReader()`. It is to get you started if you want to go in this direction rather than having to parse everything yourself. You can read more about it here: CSV File Reading and Writing, or here: Working with csv files in Python.

### Testing the Clinic Class

**Loads `simple_patients_2.csv`:**

```
from clinic import Clinic

c = Clinic("FitTrack Clinic")
c.load_csv("simple_patients_2.csv")

p = c.get_patient("P002")
print(len(p.get_session("session_001").get_log()))
```

**Output:**

```
Skipping invalid vital reading for P002 (session_001, 2024-03-12 09:
4
```

**Partial, case-insensitive search:**

```
from clinic import Clinic

c = Clinic("FitTrack Clinic")
c.load_csv("patients.csv")

print(c.search_patients_by_name("bueller"))
print(c.search_patients_by_name("nobody"))
```

**Output:**

```
[Patient('P003', 'Jeanie Bueller', 55, {'asthma', 'hypertension'})]
[]
```

**Union of all conditions, then find patients with ALL conditions:**

```
from clinic import Clinic

c = Clinic("FitTrack Clinic")
c.load_csv("patients.csv")

print(c.get_all_conditions())
```

```
print(c.get_patients_with_conditions({"hypertension", "diabetes"}))
print(c.get_patients_with_conditions({"hypertension", "asthma"}))
print(c.get_patients_with_conditions({"asthma", "diabetes"}))
```

**Output:**

```
{'hypertension', 'diabetes', 'asthma'}
{'P002'}
{'P003'}
set()
```

> ⚠ Reminder: sets are unordered, so the order of elements may vary.

**Returns the average heart rate per genre across all sessions clinic-wide:**

```
from clinic import Clinic

c = Clinic("FitTrack Clinic")
c.load_csv("patients.csv")

averages = c.get_genre_hr_averages()
for genre, avg in sorted(averages.items()):
    print(genre + ": " + str(round(avg, 1)) + " bpm")
```

**Output:**

```
hip-hop: 83.3 bpm
pop: 89.0 bpm
rock: 87.3 bpm
```

▲ Back to Top

---

## Part 5: main.py — Interactive Menu

Your `main.py` file must present an interactive text menu that loops until the user chooses to quit. Your menu must handle invalid input gracefully. If the user enters something unexpected, print a clear message and prompt again rather than crashing.

Much of `main.py` is provided. There are comments in the provided template telling you what must be added. Refer to the output provided in the **Example interactions** section below to print the correct output when both valid and invalid input are provided.

| Option | Action |
|---|---|
| 1 | Import seed data from a file (catch `FileNotFoundError` and ask for a different file name) |
| 2 | Add a new patient manually |
| 3 | Add a workout session to a patient |
| 4 | Add a song to a session's playlist |
| 5 | Log a vital reading for a session (catch `ValueError` and print a message if values are invalid) |
| 6 | View all patients |
| 7 | View all sessions for a patient (catch `ValueError`) |
| 8 | View flagged abnormal readings for a session |
| 9 | View a patient's heart rate trend across sessions |
| 10 | View all distinct conditions across the clinic |
| 11 | View patients who have **all** of a given set of conditions |
| 12 | View clinic-wide genre heart rate averages |
| 13 | Export a patient report to a `.txt` file (catch `ValueError`) |
| 14 | Quit |

## Example interactions:

### *Option 1: File not found*

```
Input selection (1-14): 1
Enter filename: wrong_file.csv
Error: File not found: wrong_file.csv
```

### *Option 1: Load patients from CSV*

```
Input selection (1-14): 1
Enter filename: simple_patients_1.csv
Patients loaded from simple_patients_1.csv.
```

### *Option 2: Add a new patient*

```
Input selection (1-14): 2
Enter patient ID: P009
Enter patient name: Allison Reynolds
Enter patient age: 39
Enter conditions (comma-separated, or leave blank): migraines
Patient added: Allison Reynolds (P009) | Age: 39 | Conditions: migraines
```

### *Option 3: Add a workout session*

```
Input selection (1-14): 3
Enter patient ID: P009
Enter session ID: session_001
Enter session date (YYYY-MM-DD): 2024-03-10
Session added: session_001 on 2024-03-10: 0 songs, 0 readings
```

### *Option 4: Add a song to a session*

```
Input selection (1-14): 4
Enter patient ID: P009
Enter session ID: session_001
Enter song title: Scenario
Enter artist name: A Tribe Called Quest
Enter BPM: 93
Enter genre: hip-hop
Song added: "Scenario" by A Tribe Called Quest (93 bpm, hip-hop)
```

### *Option 5: Valid vital reading*

```
Input selection (1-14): 5
Enter patient ID: P009
Enter session ID: session_001
Enter heart rate: 88
Enter systolic BP: 118
```

```
Enter diastolic BP: 76
Enter O2 saturation: 98.2
Enter timestamp (YYYY-MM-DD HH:MM): 2024-03-10 09:00
Vital reading added.
```

### *Option 5: Invalid vital reading*

```
Input selection (1-14): 5
Enter patient ID: P009
Enter session ID: session_001
Enter heart rate: 400
Enter systolic BP: 118
Enter diastolic BP: 76
Enter O2 saturation: 98.2
Enter timestamp (YYYY-MM-DD HH:MM): 2024-03-10 09:00
Error: Invalid heart rate: 400. Must be between 1 and 300.
Vital reading not added.
```

### *Option 6: View all patients*

```
Input selection (1-14): 6
Kevin McCallister (P001) | Age: 29 | Conditions: asthma
Allison Reynolds (P009) | Age: 39 | Conditions: migraines
```

### *Option 7: View sessions for a patient*

```
Input selection (1-14): 7
Enter patient ID: P001
session_001 on 2024-03-10: 4 songs, 4 readings
```

### *Option 7: Patient not found*

```
Input selection (1-14): 7
Enter patient ID: P999
Error: Patient not found: P999
```

### *Option 8: View abnormal readings*

```
Input selection (1-14): 8
Enter patient ID: P001
```

```
Enter session ID: session_001
No abnormal readings in this session.
```

### *Option 9: View heart rate trend*

```
Input selection (1-14): 9
Enter patient ID: P001
2024-03-10: 90.8 bpm
```

### *Option 10: View all conditions*

```
Input selection (1-14): 10
All conditions in the clinic: {'asthma', 'migraines'}
```

### *Option 11: Patients with all given conditions*

```
Input selection (1-14): 11
Enter conditions separated by commas: asthma
Patients with all of {'asthma'}: {'P001'}
```

### *Option 12: Genre heart rate averages*

```
Input selection (1-14): 12
Clinic-wide genre heart rate averages:
  pop: 98.0 bpm
  rock: 88.3 bpm
```

### *Option 13: Export patient report*

```
Input selection (1-14): 13
Enter patient ID: P009
Report exported to report_P009.txt.
```

### *Option 13: Patient not found*

```
Input selection (1-14): 13
Enter patient ID: P999
Error: Patient not found: P999
```

### *Option 14: Quit*

```
Input selection (1-14): 14
Goodbye!
```

***Green text*** *is user input,* ***white text*** *is program output.*

## Part 6: Patient Report Format

The `export_patient_report()` method in `Clinic` must write a plain text file. The file must be readable by a non-programmer. Below is an example of what the report should look like. You may adjust the exact formatting as long as all required information is present and clearly labelled.

> ⚠ Round all floating-point values to one decimal place.

The report must include:

- Patient name, ID, age, and conditions
- For each session: date, playlist, average heart rate, and any abnormal readings
- Heart rate trend across sessions (sorted oldest to newest)
- Most frequent genre across all sessions
- The session with the most abnormal readings

### Example Generated from Example Interactions in Part 5

```
============================================================
FITTRACK CLINIC — PATIENT REPORT
============================================================
Patient: Allison Reynolds
ID: P009
Age: 39
Conditions: migraines
------------------------------------------------------------

SESSION: session_001  |  Date: 2024-03-10
  Playlist:
```

```
    1. "Scenario" by A Tribe Called Quest (93 bpm, hip-hop)
  Genres this session: hip-hop
  Average heart rate: 88.0 bpm
  Abnormal readings: none

------------------------------------------------------------
SUMMARY
------------------------------------------------------------
Overall average heart rate: 88.0 bpm
Most frequent genre: hip-hop
Worst session (most abnormal readings): session_001

Heart rate trend (oldest to newest):
  2024-03-10: 88.0 bpm
============================================================
```

### More Detailed Example Using `patients.csv`

```
============================================================
FITTRACK CLINIC — PATIENT REPORT
============================================================
Patient: Kevin McCallister
ID: P001
Age: 29
Conditions: asthma
------------------------------------------------------------

SESSION: session_001  |  Date: 2024-03-10
  Playlist:
    1. "Eye of the Tiger" by Survivor (109 bpm, rock)
    2. "Mr. Brightside" by The Killers (148 bpm, rock)
    3. "Seven Nation Army" by The White Stripes (124 bpm, rock)
    4. "Don't Stop the Music" by Rihanna (171 bpm, pop)
  Genres this session: pop, rock
  Average heart rate: 90.8 bpm
  Abnormal readings: none

SESSION: session_002  |  Date: 2024-03-17
```

```
  Playlist:
    1. "Shake It Off" by Taylor Swift (160 bpm, pop)
    2. "Levitating" by Dua Lipa (103 bpm, pop)
    3. "Uptown Funk" by Bruno Mars (115 bpm, pop)
    4. "Gangnam Style" by Psy (104 bpm, pop)
  Genres this session: pop
  Average heart rate: 87.2 bpm
  Abnormal readings: none

SESSION: session_003  |  Date: 2024-03-24
  Playlist:
    1. "Golden" by HUNTR/X (155 bpm, pop)
    2. "Scenario" by A Tribe Called Quest (93 bpm, hip-hop)
    3. "Sabotage" by The Beastie Boys (124 bpm, hip-hop)
    4. "Hey Ya!" by Outkast (94 bpm, hip-hop)
  Genres this session: hip-hop, pop
  Average heart rate: 86.5 bpm
  Abnormal readings: none

------------------------------------------------------------
SUMMARY
------------------------------------------------------------
Overall average heart rate: 88.2 bpm
Most frequent genre: pop
Worst session (most abnormal readings): session_001

Heart rate trend (oldest to newest):
  2024-03-10: 90.8 bpm
  2024-03-17: 87.2 bpm
  2024-03-24: 86.5 bpm
============================================================
```

▲ Back to Top

---

## Part 7: What to Submit

Submit all of the following files. Missing files will result in marks being deducted.

- `song.py`: the `Song` class
- `vitals.py`: the `Vitals` class

- `workout.py`: the `Workout` class
- `patient.py`: the `Patient` class
- `clinic.py`: the `Clinic` class
- `main.py`: the interactive menu and program entry point

> ⚠ Do **not** submit the provided data files (`simple_patients_1.csv`, `simple_patients_2.csv`, `patients.csv`). The autograder has its own files for testing your submission.

▲ Back to Top

---

## Part 8: Starter Templates

Use the following starter templates for each file. Do not rename the classes, methods, nor parameters. Remove the `pass` keyword once you have written the code for each method.

### `song.py`

```
# Name:
# Student #:
# Username:
# Date:
# Description: Defines the Song class, which represents a single music
#              track in a workout session playlist.

class Song:

    def __init__(self, title, artist, bpm, genre):
        pass

    def get_title(self):
        pass

    def get_artist(self):
        pass

    def get_bpm(self):
        pass
```

```python

    def get_genre(self):
        pass

    def __str__(self):
        pass

    def __repr__(self):
        pass

    def __eq__(self, other):
        pass
```

## vitals.py

```python
# Name:
# Student #:
# Username:
# Date:
# Description: Defines the Vitals class, which represents one set
#              of vital signs recorded during a workout session.

class Vitals:

    def __init__(self, heart_rate, blood_pressure, o2_saturation, time
        # Validates all values against the valid ranges in Part 3.
        # Raises ValueError if any value is out of range.
        # Check order: heart_rate, systolic BP, diastolic BP, o2_satur
        # Then initializes all private attributes.
        # blood_pressure must be stored as a tuple.
        pass

    def get_heart_rate(self):
        pass

    def get_blood_pressure(self):
        pass

    def get_o2_saturation(self):
        pass

```

```
    def get_timestamp(self):
        pass

    def is_abnormal(self):
        # Returns True if any value is outside the normal range (see P
        pass


    def __str__(self):
        # Returns: "HR: <hr> bpm | BP: <sys>/<dia> | O2: <o2>% | <time
        pass

    def __repr__(self):
        pass

    def __eq__(self, other):
        # Returns True if all four values match.
        pass

    def __lt__(self, other):
        # Returns True if this reading's heart rate is lower than the
        pass

    def __gt__(self, other):
        # Returns True if this reading's heart rate is greater than th
        pass

```

## workout.py

```
# Name:
# Student #:
# Username:
# Date:
# Description: Defines the Workout class, which represents a single
#              workout session containing a playlist and vital reading

from song import Song
from vitals import Vitals

class Workout:
```

```python

    def __init__(self, session_id, date):
        # Initialises session_id and date.
        # _playlist and _log start as empty lists.
        pass

    def get_session_id(self):
        pass

    def get_date(self):
        pass

    def get_playlist(self):
        # Returns a copy of the playlist.
        pass

    def get_log(self):
        # Returns a copy of the vitals log.
        pass

    def add_song(self, song):
        pass

    def add_vital(self, vitals):
        pass

    def get_average_heart_rate(self):
        # Returns the average heart rate, or None if the log is empty.
        pass

    def get_unique_genres(self):
        # Returns a set of unique genre strings from the playlist.
        pass

    def get_abnormal_readings(self):
        # Returns a list of Vitals objects for which is_abnormal() is
        pass

    def get_segments(self):
        # Divides the vitals log into equal segments, one per song.
        # Returns a list of (Song, [Vitals]) tuples.
        # Returns None if the playlist or log is empty.
        pass

```

```python

    def get_peak_heart_rate_song(self):
        # Returns the Song with the highest average HR segment (see Pa
        # Returns None if playlist or log is empty.
        pass

    def __str__(self):
        # Format: "<session_id> on <date>: <n> songs, <n> readings"
        pass

    def __repr__(self):
        pass

    def __eq__(self, other):
        # Two Workouts are equal if their session_id matches.
        pass
```

## patient.py

```python
# Name:
# Student #:
# Username:
# Date:
# Description: Defines the Patient class, which represents a clinic pa
#              and owns a collection of Workout objects.

from workout import Workout

class Patient:

    def __init__(self, patient_id, name, age, conditions):
        # Initialises all attributes.
        # conditions must be stored as a set.
        pass

    def get_patient_id(self):
        pass

    def get_name(self):
        pass

```

```
    def get_age(self):
        pass

    def get_conditions(self):
        # Returns a copy of the conditions set.
        pass

    def get_sessions(self):
        # Returns a copy of the sessions dictionary.
        pass

    def set_name(self, name):
        pass

    def set_age(self, age):
        # Raises ValueError if age is not a positive integer.
        pass

    def add_condition(self, condition):
        pass

    def remove_condition(self, condition):
        pass

    def add_session(self, session):
        pass

    def get_session(self, session_id):
        # Returns the Workout with the given session_id, or None.
        pass

    def get_overall_avg_heart_rate(self):
        # Returns the average HR across all readings in all sessions,
        pass

    def get_heart_rate_trend(self):
        # Returns a sorted list of (date, avg_heart_rate) tuples.
        # Sessions with no vitals are skipped.
        pass

    def get_worst_session(self):
        # Returns the session with the most abnormal readings.
        # Returns None if no sessions exist.

```

```
        pass

    def get_top_genre(self):
        # Returns the most frequent genre across all sessions, or None
        pass

    def __str__(self):
        # Format: "<name> (<id>) | Age: <age> | Conditions: <condition
        # Conditions sorted alphabetically and separated by commas.
        pass

    def __repr__(self):
        pass

    def __eq__(self, other):
        # Two Patients are equal if their patient_id matches.
        pass
```

## clinic.py

```
# Name:
# Student #:
# Username:
# Date:
# Description: Defines the Clinic class, which manages the full patien
#              roster and handles all file input and output.

import csv
from patient import Patient
from workout import Workout
from song import Song
from vitals import Vitals

class Clinic:

    def __init__(self, name):
        pass

    def get_name(self):
        pass

```

```
    def set_name(self, name):
        pass

    def get_patients(self):
        # Returns a copy of the patients dictionary.
        pass

    def add_patient(self, patient):
        pass

    def get_patient(self, patient_id):
        # Returns the Patient with the given ID.
        # Raises ValueError if not found.
        pass

    def search_patients_by_name(self, name):
        # Returns a list of Patients whose name contains the given str
        # (case-insensitive, partial match).
        pass

    def get_patients_with_condition(self, condition):
        # Returns a set of patient_id strings for patients who have th
        pass

    def get_all_conditions(self):
        # Returns the union of all patients' condition sets.
        pass

    def get_patients_with_conditions(self, conditions):
        # Returns a set of patient_id strings for patients whose condi
        # set contains ALL of the given conditions (superset check).
        pass

    def get_highest_avg_hr_patient(self):
        # Returns the Patient with the highest overall average heart r
        pass

    def get_genre_hr_averages(self):
        # Returns a dict mapping genre to average HR across the whole
        # Uses the positional approximation from Part 5.
        pass

    def load_csv(self, file_name):

```

```
        # Reads the CSV file and populates the clinic.
        # Catches ValueError, prints a message, and continues.
        pass

    def export_patient_report(self, patient_id, file_name):
        # Exports a plain text report for the given patient.
        # Raises ValueError if the patient does not exist.
        pass

    def __str__(self):
        # Format: "<name> | <n> patients"
        pass

    def __repr__(self):
        pass

    def __eq__(self, other):
        # Two Clinics are equal if their name matches.
        pass
```

## main.py

```
# Name:
# Student #:
# Username:
# Date:
# Description: Main program for the Workout Vital Signs Tracker.

from clinic import Clinic
from song import Song
from vitals import Vitals
from workout import Workout
from patient import Patient


def get_menu_selection():
    print("\nSelect an option:")
    print(" 1) Import patients from a CSV file")
    print(" 2) Add a new patient")
    print(" 3) Add a workout session to a patient")
    print(" 4) Add a song to a session playlist")
```

```
    print(" 5) Log a vital reading for a session")
    print(" 6) View all patients")
    print(" 7) View sessions for a patient")
    print(" 8) View abnormal readings for a session")
    print(" 9) View a patient heart rate trend")
    print("10) View all conditions in the clinic")
    print("11) View patients with all of a set of conditions")
    print("12) View clinic-wide genre heart rate averages")
    print("13) Export patient report to .txt file")

    print("14) Quit")
    while True:
        selection = input("\nInput selection (1-15): ").strip()
        if selection.isdigit() and 1 <= int(selection) <= 14:
            return int(selection)
        else:
            print("Invalid input. Try again.")


clinic = Clinic("FitTrack Clinic")

while True:
    selection = get_menu_selection()

    if selection == 1:
        file_name = input("Enter filename: ").strip()
        # You must call load the csv here
        # Catch the error if the file isn't found


    elif selection == 2:
        patient_id = input("Enter patient ID: ").strip()
        name = input("Enter patient name: ").strip()
        age = input("Enter patient age: ").strip()
        conditions_input = input("Enter conditions (comma-separated, o
        conditions = set()
        if conditions_input:
            for c in conditions_input.split(","):
                if c.strip():
                    conditions.add(c.strip())
        # Create the Patient and add them to the clinic
        # Catch ValueErrors
        # Output should match examples from the assignment sheet

```

```


    elif selection == 3:
        patient_id = input("Enter patient ID: ").strip()
        session_id = input("Enter session ID: ").strip()
        date = input("Enter session date (YYYY-MM-DD): ").strip()
        # Find the patient associated with the clinic
        # Add the new Workout session to the patient
        # Catch ValueErrors
        # Output should match examples from the assignment sheet


    elif selection == 4:
        patient_id = input("Enter patient ID: ").strip()
        session_id = input("Enter session ID: ").strip()
        title = input("Enter song title: ").strip()
        artist = input("Enter artist name: ").strip()
        bpm = input("Enter BPM: ").strip()
        genre = input("Enter genre: ").strip()
        # Find the patient associated with the clinic
        # Find the session associated with the patient
        # Add the new Song to the session
        # Catch ValueErrors
        # Output should match examples from the assignment sheet


    elif selection == 5:
        patient_id = input("Enter patient ID: ").strip()
        session_id = input("Enter session ID: ").strip()
        heart_rate = input("Enter heart rate: ").strip()
        systolic = input("Enter systolic BP: ").strip()
        diastolic = input("Enter diastolic BP: ").strip()
        o2 = input("Enter O2 saturation: ").strip()
        timestamp = input("Enter timestamp (YYYY-MM-DD HH:MM): ").stri
        # Find the patient associated with the clinic
        # Find the session associated with the patient
        # Add the new Vital to the session
        # Catch ValueErrors
        # Output should match examples from the assignment sheet


    elif selection == 6:
        # Print off all patients from the clinic
```

```
        # If there aren't any, print "No patients in the clinic."
        pass

    elif selection == 7:
        patient_id = input("Enter patient ID: ").strip()
        # Find the patient associated with the clinic
        # Find all the session(s) associated with the patient
        # Catch ValueErrors
        # Output should match examples from the assignment sheet


    elif selection == 8:
        patient_id = input("Enter patient ID: ").strip()
        session_id = input("Enter session ID: ").strip()
        # Find the patient associated with the clinic
        # Find the session associated with the patient
        # Catch ValueErrors
        # Output should match example from the assignment sheet if no
        # Output should show all abnormal readings if there are any


    elif selection == 9:
        patient_id = input("Enter patient ID: ").strip()
        # Find the patient associated with the clinic
        # Find the patient's heart rate trend
        # Catch ValueErrors
        # If there are no readings, print "No readings available for <
        # Otherwise, output should show the heart rate trend as in the


    elif selection == 10:
        # Print all of the conditions associated with the clinic as in
        # If no conditions, print "No conditions recorded in the clini
        pass

    elif selection == 11:
        conditions_input = input("Enter conditions separated by commas
        # Print all of the patients with the conditions
        pass

    elif selection == 12:
        # If there is no data, print "No genre data available."
        # Otherwise, print the averages by genre as in the example fro
```

```
            pass

        elif selection == 13:
            patient_id = input("Enter patient ID: ").strip()
            file_name = "report_" + patient_id + ".txt"
            # Export the patient's information to a txt file
            # Catch ValueErrors

        elif selection == 14:
            print("Goodbye!")
            break
```

▲ Back to Top

## Non-Functional Specification and Other Details

In addition to the other requirements given in the previous sections, your program must also fulfill the following requirements:

- **Document each file:** You MUST provide a comment at the top of **all** Python files that contains 1) your full name, 2) your student number, 3) your UWO username, 4) the date the file was created, and 5) a description of what this specific file does. The description must be different in each file.
- **Comment each method:** Before every method definition, include a comment describing what the method does, what its parameters are, and what it returns (if anything).
- **Other comments throughout your code:** You must also include comments for any lines that are not easily understood. Use good judgment — the marker may remove marks for over or under commenting.
- **No code outside class definitions:** Each class file should contain only the class definition and its import statements. Do not put any testing code in these files. Testing code belongs in `main.py`.
- **Meaningful variable names:** Use descriptive names in snake_case. Do not use single-letter names except as loop counters.
- **No global variables:** All communication between methods must happen through parameters and return values. You may not use the `global` keyword.
- **No external modules:** You may only import from the Python standard library (`csv`, `os`). Do not import any third-party packages.

- **No hardcoding:** Your program must work correctly for any valid input. Attempts to fool or trick the autograder will result in a zero grade for that test.
- **Follow good style:** As much as possible, follow the PEP8 style guide. Both the autograder and your IDE should provide hints about style violations.
- **Python 3.12:** Your code should work correctly in Python 3.12 and newer.

▲ Back to Top

## Assessment

The assignment will be marked as a combination of auto-graded tests and manual grading of your code logic, comments, formatting, and style. Marks will be deducted for failing to follow any of the specifications in this document, not documenting your code with comments, using poor formatting or style, hardcoding, or naming your files incorrectly.

**Marking Scheme:**

- **Autograded Tests:** 80 points
- **Header comment including your name, student ID, username, creation date, and description of file in all files:** 4 points
- **Descriptive in-line comments throughout code:** 4 points
- **Comment for each method describing what it does, its parameters, and its return values (if any):**4 points
- **Meaningful variable names:** 2 points
- **Good programming practices *(see Non-Functional Specification)*:** 6 points

**Total: 100 points**

***Note:*** *Marks from the autograded tests may also be manually removed by the marker for violating any of the requirements given in the Non-Functional Specification, hard coding solutions, or failing to implement any features as specified.*

▲ Back to Top

© 2026 CS1026B